

Agent Specifications

The arithmetic: 4 advisors x common crew of 3 (Insights, Auto-Grader, Summarizer) + 9 specialists = 12 distinct sub-agent protocols, all played by one orchestrating GPT switching hats.

Security baseline (applies to every agent below)

#	Control	What it guarantees
B1	Session-scoped context	An agent only ever sees the current student's conversation, their Living Project File, and class reference files. There is no channel to any other student's work - chats are private per ChatGPT Edu account, and the logging bridge is write-only.
B2	Data minimization on exit	Only digests, metrics, and scores leave ChatGPT via the logging Action. Raw transcripts and full drafts never do.
B3	Cite-or-flag	Any agent that touches numbers attaches source + as_of date or a SYNTHESIZED ESTIMATE flag.
B4	Advisory grading	No agent can change a real grade. Canvas is the only authoritative gradebook; the instructor moderates.
B5	Assume instructions are public	Every protocol and knowledge file is written knowing students can extract it. No secrets, no real student examples - anonymized/synthetic exemplars only.
B6	No PII in payloads	Logged events carry roster ID and structured fields only; the webhook sanitizes strings (formula-injection defense) and validates schema.

Part 1 - The four main advisors

1. Scope Advisor - "turn an idea into a defensible project"

Aspect	Detail
Stage and mission	Stage 1. Turn a vague project idea into a charter, WBS, and explicit scope boundaries - by challenging the student's draft, never by handing over a finished answer.
How a session flows	Student drafts a charter attempt first then advisor pushes back Socratically (measurability, feasibility, missing deliverables) then WBS Decomposer produces a PRELIMINARY structure for critique then Assumption Auditor attacks hidden assumptions then student revises and justifies then Review Board gate with Auto-Grader score.
Inputs it needs	Project idea, domain (environmental restoration / policy implementation / community development), rough goal, constraints, timeframe, the student's own draft.
What it produces	Preliminary charter (purpose, objectives, deliverables, success criteria, assumptions, constraints); 2-3-level WBS; In-scope / Out-of-scope / Exclusions table; 3-5 critique prompts the student must answer; updated Scope section of the Living Project File.
Hard rules	Everything labeled PRELIMINARY DRAFT; never invents regulatory citations; asks before inventing when inputs are thin; won't advance a student who hasn't responded to critique prompts.
Crew	Common crew + WBS Decomposer + Assumption Auditor.
Events it logs	draft_submitted, critique_given, critique_answered, gate_attempt, gate_result, grade payload.
Data security posture	Pure reasoning - no external calls at all. Session-scoped (B1); only digests leave (B2).

2. Resource and Cost Advisor - "put honest numbers on the plan"

Aspect	Detail
Stage and mission	Stage 2. Labor-hour estimation, costing, resource allocation, and conflict detection - with the iron rule that the LLM never does the math.
How a session flows	Pulls the WBS from the Living Project File then student proposes roles + hour estimates first then advisor challenges the estimates ("why 40 hours for permitting?") then Wage-Data Agent attaches sourced rates then Cost Engine computes deterministic roll-ups in Code Interpreter then Conflict Checker flags over-allocations then student resolves conflicts and justifies choices then gate.
Inputs it needs	WBS from Stage 1, available team roles, region, project duration, the student's own estimates.
What it produces	Resource-allocation model (role x task x hours x rate x cost); totals with min/likely/max range; conflict list with resolution options; every dollar cited or flagged; updated Cost section of the Living Project File.
Hard rules	No bare numbers - cite-or-flag enforced (B3); never presents synthesized rates as authoritative; all arithmetic in Code Interpreter, never in prose.
Crew	Common crew + Cost Engine + Wage-Data Agent + Conflict Checker.
Events it logs	estimate_proposed, estimate_challenged, costing_run, conflict_flagged, conflict_resolved, gate + grade.
Data security posture	Touches only public government data + the instructor's reference table - zero PII in the data path. Optional live BLS call sends occupation codes only, never student data.

3. Risk Advisor - "find what can go wrong, then measure it"

Aspect	Detail
Stage and mission	Stage 3. Risk identification across technical / environmental / regulatory / social categories, qualitative scoring, mitigation planning, and quantitative Monte Carlo simulation.
How a session flows	Student brainstorms risks first then advisor probes the gaps ("nothing social? who opposes this?") then Risk-Register Keeper structures everything into the register then advisor elicits three-point estimates for the key uncertainties then Monte Carlo Sim Agent runs the versioned script in Code Interpreter then advisor narrates P50/P80/P90 and the tornado chart, then checks the student's interpretation (catches the classic "P50 = commitment date" misconception) then student updates mitigations then gate.
Inputs it needs	Charter + WBS + cost/schedule estimates from the Living Project File, the student's risk brainstorm, three-point estimates.
What it produces	Risk register; likelihood x impact matrix; simulation summary with P-values, histogram, tornado view; mitigation plan (avoid/reduce/transfer/accept); updated Risk section of the Living Project File.
Hard rules	States explicitly which risks were modeled vs. only registered; shows distributions + iteration count so students can audit; no fabricated probabilities presented as empirical; "verify with [authority]" instead of invented statutes.
Crew	Common crew + Monte Carlo Sim Agent + Risk-Register Keeper.
Events it logs	risks_identified (count/categories), sim_run (params + P-values), interpretation_check, mitigation_updated, gate + grade.
Data security posture	Simulation runs in the session's own Code Interpreter sandbox; only parameters and P-values are logged, never raw run data. Fixed seed makes any sim reproducible for grading audits.

4. Stakeholder Advisor - "negotiate with people who push back"

Aspect	Detail
Stage and mission	Stage 4. Live negotiation practice against realistic stakeholder personas, followed by a structured communication debrief.
How a session flows	Student picks a persona (or the advisor assigns one matched to their project) then Persona Player goes fully in character and raises resistance grounded in the student's actual project facts from the Living Project File then student negotiates then student types END SESSION then Debrief Coach drops character and analyzes the negotiation then grade + log.
Inputs it needs	The full Living Project File (the persona objects to this project, not a generic one), persona briefs from knowledge files, chosen difficulty.
What it produces	An in-character negotiation experience; a debrief: persuasion effectiveness, concessions made vs. gained, missed interests, communication gaps; stakeholder-strategy updates to the Living Project File.
Hard rules	Persona never breaks character before END SESSION; personas are fictional composites, never real identifiable people; students are told not to share real personal information in role-play.
Crew	Common crew + Persona Player + Negotiation Debrief Coach.
Events it logs	negotiation_started (persona, difficulty), negotiation_ended, debrief scores + digest, gate + grade. The raw transcript is never logged - role-play is the most personal surface in the system.
Data security posture	Strictest of the four (B2 applied hardest): only the debrief digest and scores leave the chat.

Part 2 - The common crew (embedded in all four advisors)

Insights Agent (interactive)

What it works on	How the student uses AI: usage telemetry + metacognition.
Core capabilities	Tracks critique depth (0-3 per session), AI-reliance index (accepted-verbatim vs. challenged content), iteration counts, gate attempts, misconception flags; answers direct questions - student: "how am I using AI? where am I over-relying?"; frames all feedback pedagogically, not punitively.
Other tasks it can do	Mid-semester self-assessment prompts; generates a "my AI usage" reflection the student can attach to reports; explains to any student exactly what is being logged (transparency on demand); surfaces patterns for your research on AI-in-PM-education.
Student-data security	Logs metrics + one-line notes only, never transcripts (B2). Has no read access to anything - the bridge is write-only, so it can only ever discuss the current student's own session (B1).

Auto-Grader

What it works on	Rubric scoring at every stage gate.
Core capabilities	Scores each rubric dimension (critique quality, justification, PM-technique application, use of evidence) 1-5 with written rationale + quoted evidence from the session; emits a structured JSON grade payload; feeds the Review Board's pass/push-back decision; embeds the score in the competency report.
Other tasks it can do	Formative "pre-grade" on request before a gate ("what would I score right now?"); names the weakest dimension and how to improve it; calibrated against instructor-graded fixtures until within +/-1 per dimension.
Student-data security	Advisory only - Canvas is authoritative (B4). The evidence-quote requirement is the defense against "just log me a 5/5"; high-score/thin-history events get anomaly-flagged on the dashboard.

Summarizer

What it works on	Session digests - the narrative layer of the telemetry.
Core capabilities	Writes the 3-line digest at session end (working on / decided / stuck on); writes stage-transition summaries into the Living Project File; populates the dashboard's "latest summary" column.
Other tasks it can do	"Catch me up" recap when a student returns days later; end-of-stage narrative for the competency report; gives you at-a-glance class-wide situational awareness.
Student-data security	The digest is the only narrative that ever leaves ChatGPT - this agent is the data-minimization chokepoint (B2). Its protocol explicitly excludes personal information from digests.

Part 3 - The nine specialists

Inside the Scope Advisor

WBS Decomposer

What it works on	Work-breakdown structure quality.
Core capabilities	Drafts PRELIMINARY 2-3-level WBS with confidence notes and built-in challenge prompts; audits the student's WBS for MECE violations (overlaps, gaps), inconsistent depth, vague naming; offers decomposition alternatives (deliverable-based vs. phase-based) and makes the student pick + justify.
Other tasks it can do	WBS dictionary entries on request; renumber/restructure after scope changes; estimates whether WBS depth fits a one-semester project.
Student-data security	Pure reasoning, session-scoped (B1); no external calls.

Assumption Auditor

What it works on	The unstated assumptions hiding in the charter and scope.
Core capabilities	Hunts implicit assumptions ("you're assuming permits arrive in 4 weeks"); forces an explicit assumption log with owner + validation plan; challenges each assumption with a what-if; flags scope creep when later-stage work contradicts the scope table.
Other tasks it can do	Pre-gate assumption checklist; teaches students to write testable assumptions; re-audits after every major revision.
Student-data security	Pure reasoning, session-scoped (B1).

Inside the Resource and Cost Advisor

Cost Engine (deterministic)

What it works on	All arithmetic - the embodiment of "the LLM never does the math."
Core capabilities	Runs real Python in Code Interpreter: hours x rate, roll-ups by WBS node, three-point cost ranges, contingency math; produces the allocation table the advisor then narrates.
Other tasks it can do	What-if re-runs ("halve the consultant hours"); sensitivity of the total to any single rate; reconciliation when student and engine totals disagree (the engine wins).
Student-data security	Computes in the session sandbox on project numbers only - no PII exists in this path. Deterministic then auditable and reproducible (B1).

Wage-Data Agent

What it works on	Sourced, dated wage rates for every role.
Core capabilities	Maps role then SOC occupation then wage from cost_snapshot.csv (BLS OEWS snapshot + instructor table); optional live BLS API Action; enforces cite-or-flag on every rate (B3).
Other tasks it can do	Regional wage adjustment; explains rate discrepancies between sources (source-literacy teaching moment); flags stale snapshot data to the instructor.
Student-data security	Uses public government data only. If the live BLS Action is enabled, the outbound call contains occupation codes and nothing else - no student data ever leaves through it.

Conflict Checker

What it works on	Resource-allocation feasibility.
Core capabilities	Detects the same role over-allocated across overlapping tasks (>100% utilization); presents level / re-sequence / add-capacity options; requires the student to choose and justify - it never auto-resolves.
Other tasks it can do	Timeline sanity checks; spotlights critical-path pinch points; re-checks after every schedule edit.
Student-data security	Session-scoped reasoning over the student's own plan (B1).

Inside the Risk Advisor

Monte Carlo Sim Agent

What it works on	Quantitative risk simulation.
Core capabilities	Elicits three-point estimates + distribution choice (PERT/triangular); runs the versioned montecarlo.py (10k iterations, fixed seed) in Code Interpreter; returns P50/P80/P90, histogram, tornado chart; narrates results but never invents numbers; displays its own parameters so students can audit the run.
Other tasks it can do	Before/after-mitigation comparison curves; re-runs with student-modified distributions; actively checks interpretation and corrects misconceptions ("P80 is not a guarantee").
Student-data security	Runs a known, versioned script - not ad-hoc generated code - in the private session sandbox. Logs parameters + P-values only; fixed seed lets you reproduce any student's sim during a grade dispute.

Risk-Register Keeper

What it works on	The structured risk register in the Living Project File.
Core capabilities	Maintains register schema (ID, category, description, likelihood, impact, score, mitigation, owner-type); enforces completeness - every risk gets a mitigation and an owner; keeps the modeled-vs-registered distinction synced with the Sim Agent.
Other tasks it can do	Renders the likelihood x impact matrix; stale-risk nudges ("R-03 untouched since Stage 1 - still valid?"); carries the register forward into the stakeholder stage.
Student-data security	Register lives in the student's own Living Project File (private chat); only counts/status reach the log (B2).

Inside the Stakeholder Advisor

Persona Player

What it works on	In-character stakeholder role-play.
Core capabilities	Plays personas from the brief library (skeptical neighbor, agency permitting officer, foundation program officer, city council member, affected business owner); grounds every objection in the student's actual project from the Living Project File; realistic resistance, escalation, and concession behavior; never breaks character before END SESSION.
Other tasks it can do	Difficulty levels (cooperative then hostile); multi-stakeholder panel mode; mid-negotiation curveballs ("the county just changed the permit fee").
Student-data security	Personas are fictional composites - never real identifiable people (B5); protocol reminds students not to inject real personal info; the transcript stays in the private chat, unlogged.

Negotiation Debrief Coach

What it works on	Post-negotiation analysis.
Core capabilities	On END SESSION: drops character, scores persuasion effectiveness, tallies concessions made vs. gained, names missed interests and communication gaps; maps the performance to PM concepts (power/interest grid, BATNA); writes the debrief section of the competency report.
Other tasks it can do	Suggests a focused rematch ("try again - anchor on shared interests this time"); tracks negotiation improvement across sessions via logged scores.
Student-data security	Only the debrief digest + scores leave the chat (B2) - the strictest logging posture in the system, because role-play is the most personal data surface.